

AI Agent defense architecture

▲ ● ■ Школа анализа данных

every boundary is a control point

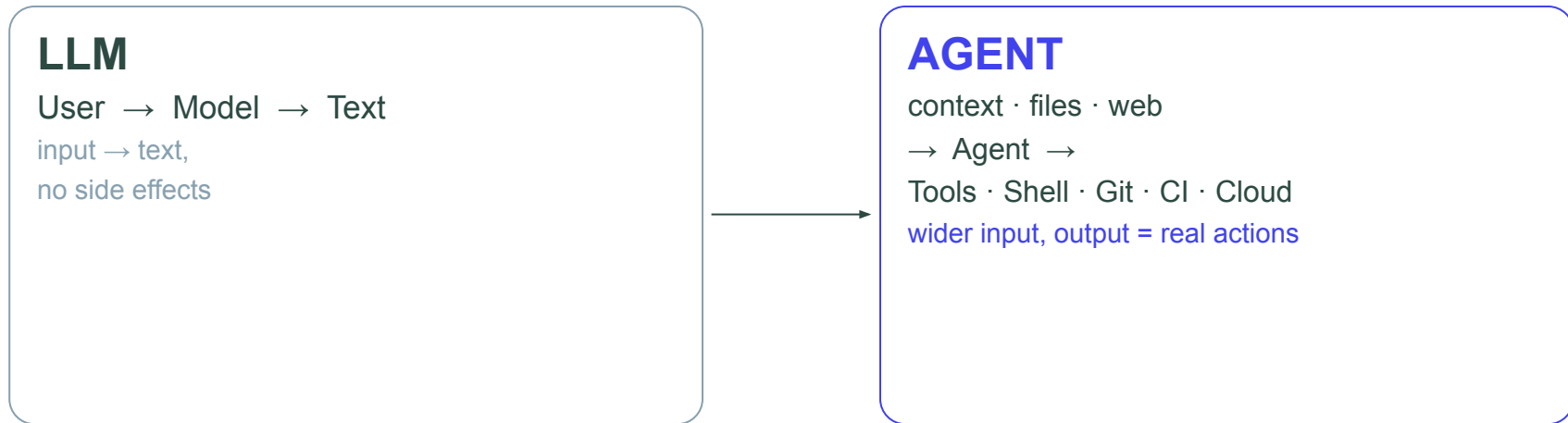
Ratmir Samarkhanov
Managed Gitlab, SourceCraft, Yandex



What we'll cover

- 01 How an agent is built
layers, boundaries, and where it's weak
- 02 Method: where to find weak spots
threat-modeling and blast-radius priority
- 03 Defense across the lifecycle
ADLC — controls built in, not bolted on at release

From LLM to agent: risk is in the action



The question shifts: not “what it said” — but “what it did”

Agent security is a property of the architecture,
not a filter

not a filter in front of the model, not a release checklist

A working definition of an agent

AGENT = LLM + context + state + tools + rules + loop

agent security = securing the **execution loop**, not just the prompt

Context

gathers its own context
from external sources?

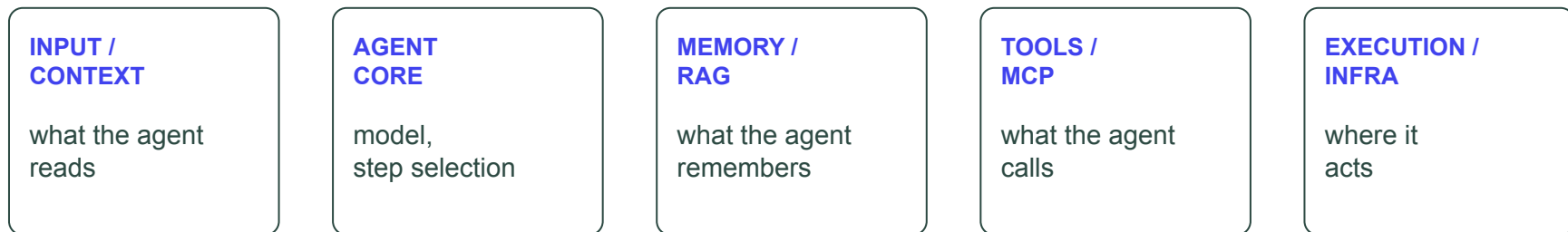
Decision

chooses the next
step itself?

Action

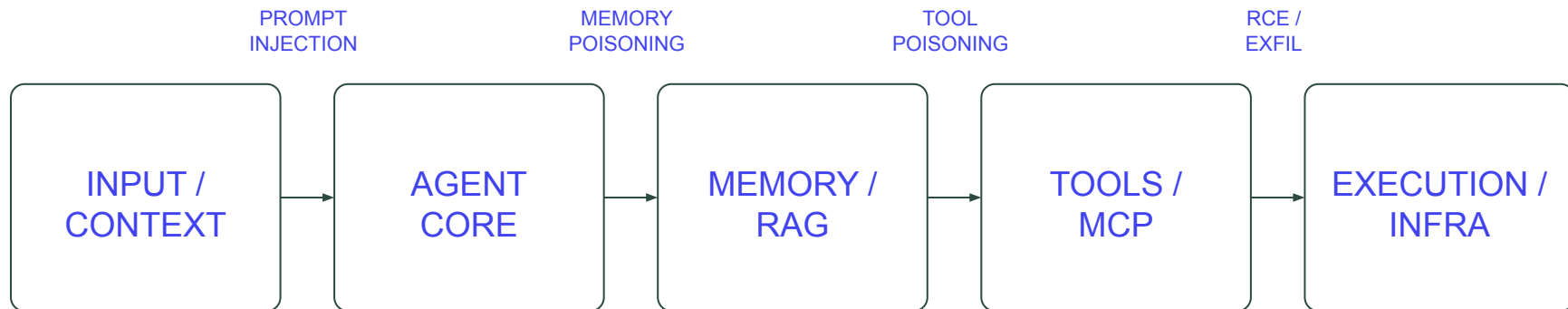
acts with real
privileges?

What an agent is made of: five layers



The model is **only one of five** layers. Each one needs defense

Security lives between the layers



The attack is **on the arrow, not in the box**

Agent → agent is a trust boundary too

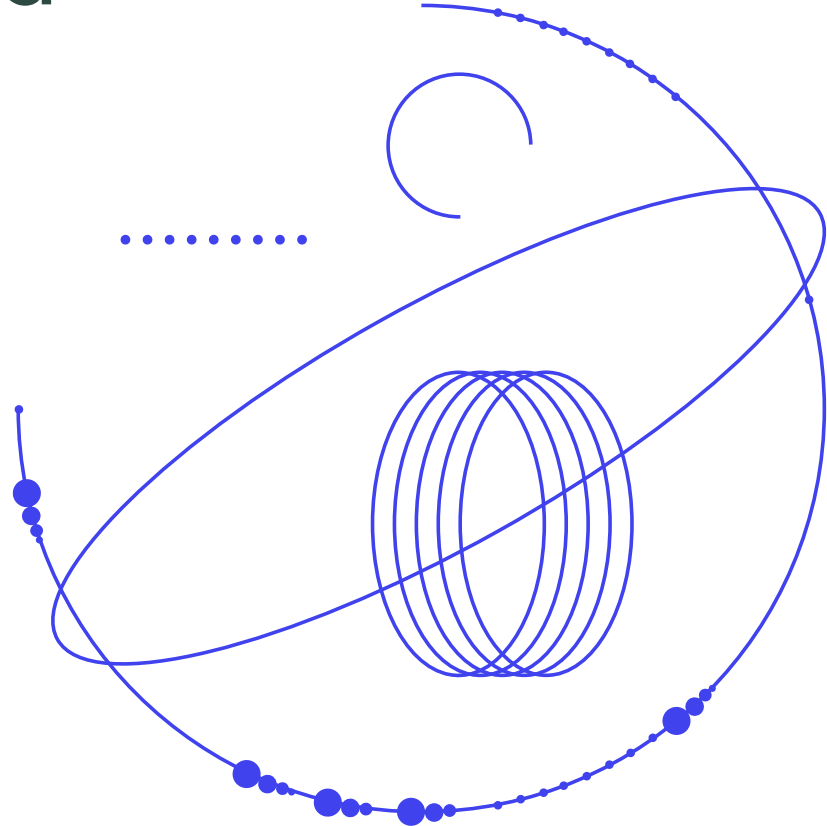
Text = data + commands

text = data and commands at once

anything from outside is data, not commands

a document, tool output, someone's PR, an email — all data, even if it says “do this right now” inside

Method: where to find weak spots

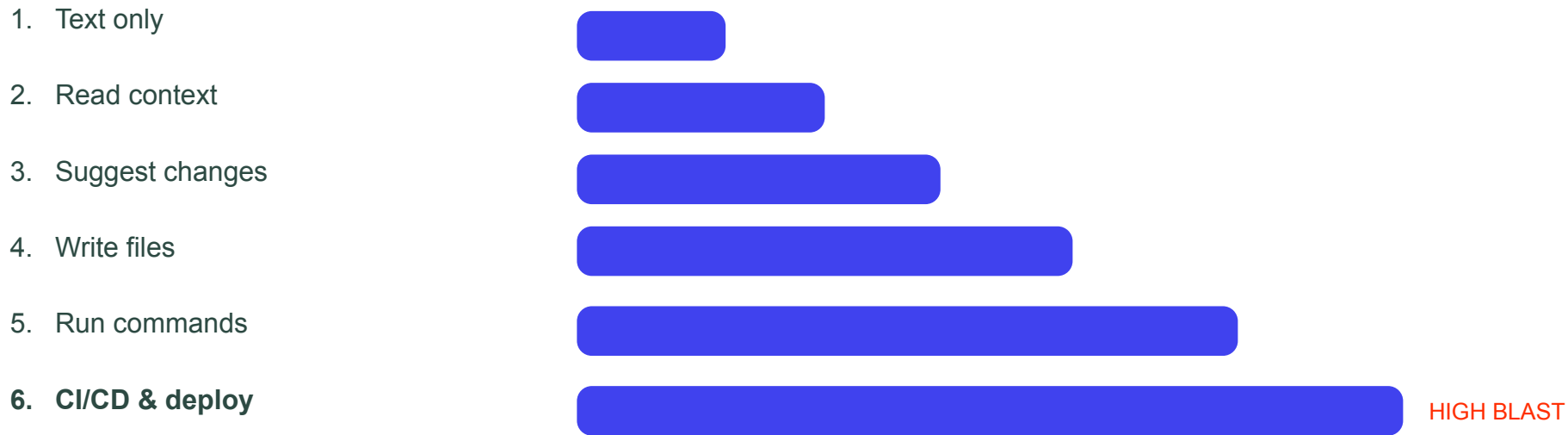


Threat-modeling an agent

1. Draw the data flow across the 5 layers
2. Mark every boundary — the arrows between layers
3. At each boundary: “what’s the worst an attacker could do?”
4. Rank the boundaries by blast radius
5. Add controls, starting with the largest radius

Then — the specifics of each step.

More capability → wider blast radius



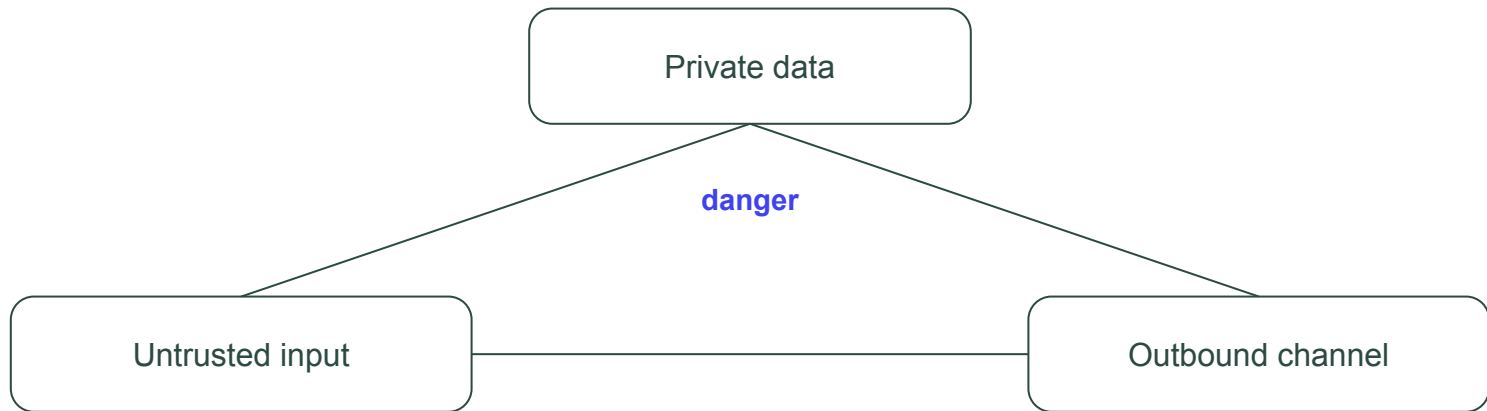
Injection = the trigger · privileges and tools = [the blast radius](#)

Blast radius → set of controls

What the agent can do	Radius	Minimum set of controls
Read-only in the working dir	low	command allowlist
Write to the working dir	↑	allowlist + human approval
Arbitrary shell / run code	high	isolation (sandbox) + approval
External APIs with data	high	short-lived narrow creds + audit
CI/CD, deploy	max	all above + review + reduced rights

Approval only works if the human understands what they're approving.

The lethal trifecta as a design rule

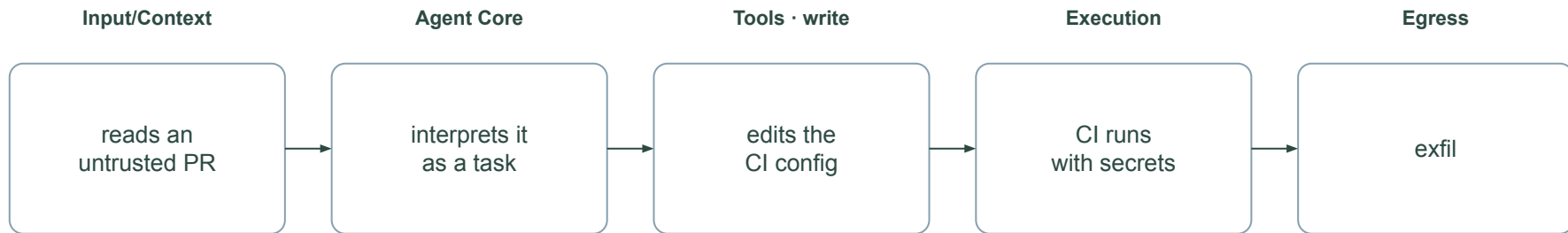


read others' PRs → no egress

hold secrets → don't accept untrusted input

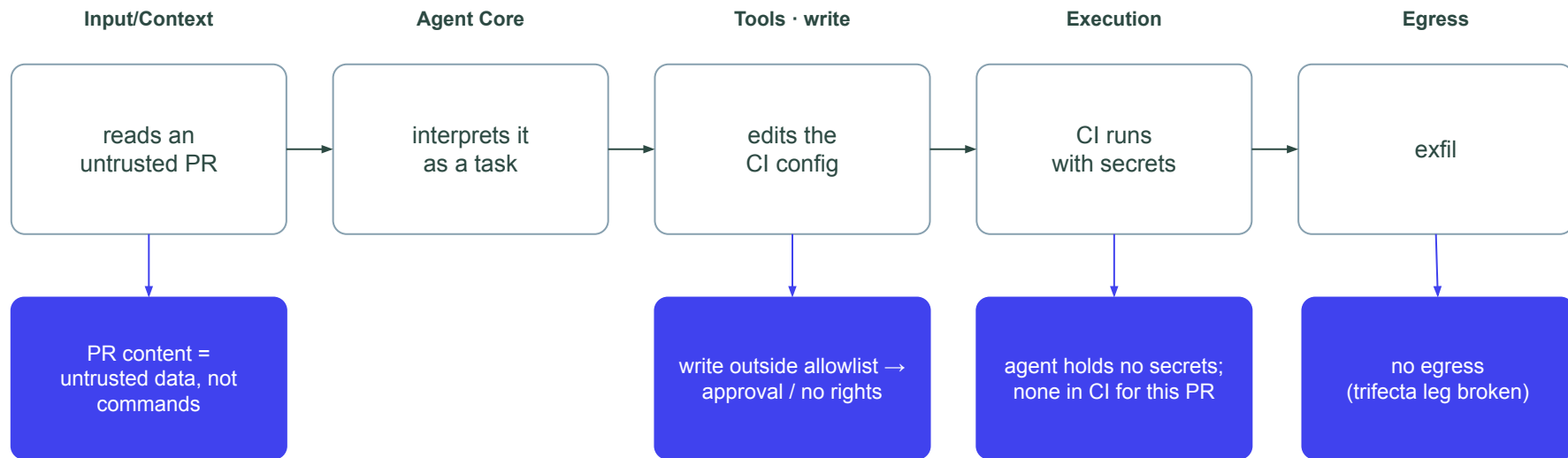
Trace: an attack chain

We build it ourselves from the 5-layer map — nothing exotic



Every step is a normal, legitimate action — nothing is “exploited” yet

Trace: where it breaks



Attacker must break every boundary — we only need to hold **one**

Confirmed in the wild

EACH ONE BREAKS A DIFFERENT LINK OF OUR CHAIN

INPUT → EGRESS

GitLab Duo

Legit Security · 2025

A hidden prompt in a merge request makes the assistant render an `<img>` and exfiltrate private source code.

AGENT CORE · TOOLS

GitHub MCP

Invariant Labs · 2025

A planted public issue makes an over-privileged agent read private repos and leak them into a public PR.

TOOLS · EXECUTION

CurXecute · Cursor

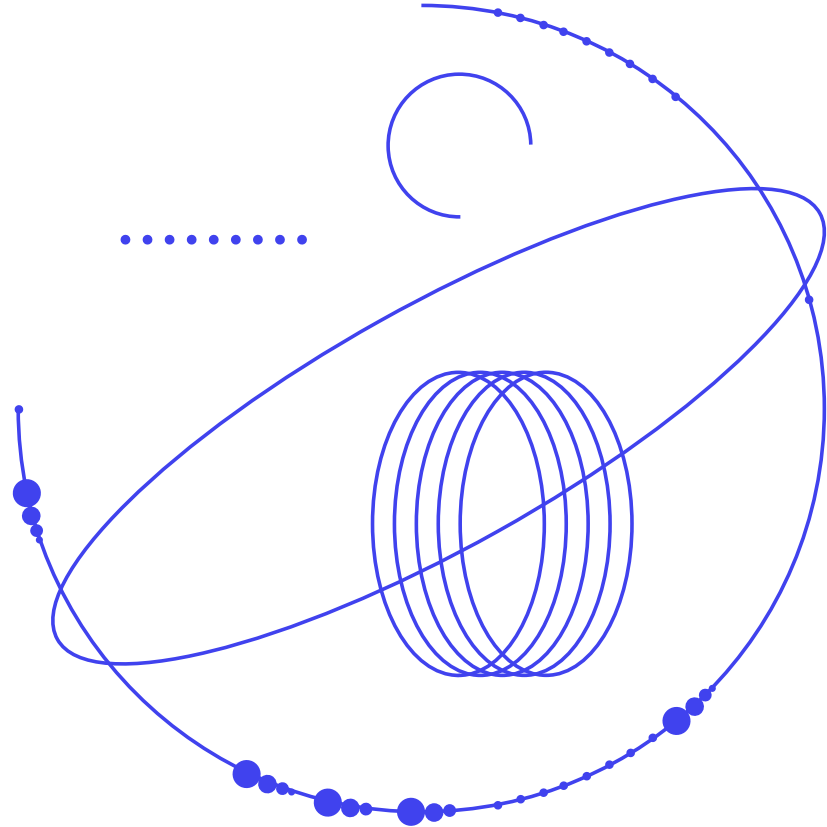
Aim Labs · CVE-2025-54135

Injection writes to `mcp.json`; the new entry auto-runs → remote code execution.

Every link of our chain has a real precedent: [Input](#) → [Core](#) → [Tools](#) → [Execution](#) → [Egress](#)

Same five boundaries — different products. Break any one link and the chain stops

Lifecycle



Not just code — behavior too

	SDLC · software	ADLC · agent
Artifact	commits, versions, deploys	instructions, context, policies
What changes	code, configs, dependencies	data, memory, tools, privileges
Boundary	who commits what	who can act with which privileges
Feedback	bugs, tests, review	logs, traces → new constraints

Goal — keep a model mistake from [becoming a compromise](#)

Eight stages of ADLC

01
Design

boundaries; what's off-limits

02
Tools

allowlist; description = trust

03
Context

provenance; external = data

04
Memory

check on the way in / out

05
Identity

the agent never sees the
secret

06
Gateway

auth, metadata, logging

07
Red Team

attack yourself, regularly

08
Observe

log enough to investigate

A control belongs to a stage, not to a list of tips

What to log for an agent

08 Observe

the prompt

request identity / token

tool calls + arguments

model version

retrieved RAG chunks

routing decisions

guardrail verdicts

If the agent is breached and there's no trail, you can't investigate

An agent's log is wider than a normal backend log — it's part of defense

Privileges are a system property

05 Identity

Anti-pattern

A raw key — right in the agent's hands

Bearer sk_live_...

compromise = leaked key
one key with rights to everything

Pattern

Agent sees `call($PROFILE_DB_RO)`



System injects the token · logs it



External DB · Bearer sk_live_...

scoped · short-lived · per actor

What's allowed is decided by the **system**, not the prompt

Anti-patterns: don't do this

- Securing only the LLM, not the boundaries
- One system prompt for everyone
- Raw keys right in the agent
- Granting rights via the prompt, not the system
- “Internal agent = safe”
- “OK” on a long script you didn’t read
- An open MCP endpoint — “no one will reach it”
- Set up a guardrail once and forget it

Reference architecture



ADLC: Design · Tools · Context · Memory · Identity · Gateway · Red Team · Observe

Boundary architecture + lifecycle + blast-radius priority

Boundaries

a control on every arrow

Lifecycle

from Design to Observe

Radius

the most dangerous first

At its core — ordinary DevSecOps. One input just happens to be arbitrary now

Thank you for your attention

Ratmir Samarkhanov
Managed Gitlab, SourceCraft, Yandex

